

Système de planification intelligente de tâches avec dépendances

Cahier des charges

Martinet Rémy · Prigot Paul · Bouaziz Ayoub · Wattiez Mathilde

L2 Informatique IED

Version : 1.0 · Date : 16 février 2026

1. Contexte et présentation du projet	3
2. Objectifs du projet	3
2.1 Objectifs fonctionnels	3
2.2 Objectifs pédagogiques	4
3. Périmètre du projet	5
3.1 Fonctionnalités incluses	5
3.2 Fonctionnalités exclues	5
3.3 Fonctionnalités optionnelles	6
4. Choix techniques	7
4.1 Technologies utilisées	7
4.2 Justification des choix	7
5.1 Architecture cible	8
5.2 Modélisation des données (format JSON)	8
5.3 Algorithmes et règles de calcul	9
5.4 API (Flask léger)	10
6. Fonctionnement général	11
7. Répartition du travail	12
8. Tests et validation	13
8.1 Stratégie de tests	13
8.2 Validation du projet	13
9. Livrables attendus	14

1. Contexte et présentation du projet

L'objectif est de réaliser une application capable de planifier automatiquement des tâches avec des dépendances, en prenant en compte des contraintes simples telles que la durée et la priorité des tâches. Ce type de planification est utile dans tous les contextes, qu'ils soient professionnels, personnels ou académiques. L'application vise à fournir un outil permettant à tout utilisateur de gérer efficacement des ensembles de tâches interconnectées. Elle garantit un ordre d'exécution logique tout en détectant automatiquement les incohérences.

Chaque tâche est caractérisée par plusieurs paramètres :

- Durée estimée : le temps nécessaire à son exécution.
- Dépendances : la liste des tâches qui doivent être terminées avant son lancement.
- Priorité : un niveau d'importance permettant d'influencer l'ordre d'exécution parmi les tâches indépendantes.

Le projet comprendra également une interface utilisateur permettant de visualiser le planning sous forme de liste ordonnée et de diagramme de Gantt simplifié, facilitant la lecture et le suivi des tâches. L'outil doit rester simple, intuitif et robuste, afin d'être utilisé sans formation particulière et de fonctionner sans configuration complexe sur un environnement Linux (Kubuntu).

2. Objectifs du projet

2.1 Objectifs fonctionnels

L'application devra permettre à l'utilisateur de :

- Créer, modifier et supprimer des tâches.
- Définir des dépendances entre les tâches.
- Calculer automatiquement un ordre d'exécution valide des tâches, en respectant les dépendances et les priorités définies.
- Détecter les dépendances cycliques et générer des alertes pour l'utilisateur.
- Calculer les dates de début et de fin de chaque tâche à partir de leur ordre d'exécution.
- Afficher le planning de manière lisible, à la fois sous forme de liste ordonnée et de mini diagramme de Gantt.

2.2 Objectifs pédagogiques

Ce projet constitue une application concrète de plusieurs notions étudiées dans le cadre de la Licence :

- Graphes et algorithmes de tri topologique : mise en pratique pour gérer les dépendances entre tâches.
- Structuration d'une application complète : conception d'un backend pour gérer les traitements et d'un frontend pour l'interface utilisateur.
- Gestion des données : utilisation de fichiers JSON pour stocker et récupérer les informations des tâches.
- Travail en équipe : répartition des rôles et collaboration via un dépôt Git pour le versionnement et le suivi des modifications.
- Résolution de problèmes concrets : détection de cycles, calcul de planning, affichage clair et lisible.

Le projet permet également de développer des compétences en conception d'interface, en planification et en test logiciel, tout en consolidant la pratique de Python et la manipulation de structures de données complexes.

3. Périmètre du projet

Le projet comprend plusieurs fonctionnalités prévues, avec la possibilité d'en ajouter selon son déroulement.

3.1 Fonctionnalités incluses

Gestion de dépendances

Le programme doit appliquer un système de dépendances simples à la planification des tâches. Une tâche ne peut commencer que lorsque toutes ses dépendances sont terminées.

Exécution simultanée des tâches

Le programme doit permettre la planification de tâches en simultané, tant que celles-ci n'ont pas de dépendances entre elles.

Gestion des priorités

Le programme doit permettre à l'utilisateur d'appliquer des priorités simples aux tâches qui influenceront leur ordre d'exécution.

Interface web simple

Le programme comprendra une interface web pour l'affichage et la gestion des tâches. Les tâches seront représentées en listes ordonnées et en mini diagramme de Gantt simplifié pour un affichage efficace et compréhensible pour l'utilisateur.

3.2 Fonctionnalités exclues

Dans le but de limiter les erreurs de configuration et de garantir la simplicité d'utilisation du programme, plusieurs fonctionnalités sont exclues.

Authentification. Le programme n'inclura pas de système d'authentification pour l'utilisateur.

Aucune gestion spécifique de la sécurité applicative n'est prévue.

Gestion multi-utilisateurs

Le programme ne permettra pas l'utilisation par plusieurs utilisateurs différents sur une même machine.

3.3 Fonctionnalités optionnelles

Synthèse générée par IA

Selon le déroulement du projet, le planning généré pourra être accompagné d'une courte synthèse générée par une IA simple.

4. Choix techniques

4.1 Technologies utilisées

Le projet sera développé principalement en **Python**. Ce langage est **maîtrisé** par l'ensemble du groupe et, comme le projet repose principalement sur des **algorithmes de planification** et de **gestion des dépendances**, Python permettra une implémentation claire, lisible et directement exploitable.

Le programme reposera sur :

- **Python** pour les algorithmes de planification et la logique métier
- des **structures de données natives** (listes, dictionnaires)
- un **stockage des données** en fichiers **JSON**
- une **interface simple** :
 - soit en **ligne de commande (CLI)**,
 - soit via une **interface web**, basée sur un micro-framework Python comme **Flask**

Le projet sera réalisé en collaboration via un **dépôt commun Git**, permettant le **versionnement** et le suivi des modifications.

4.2 Justification des choix

Ces choix techniques répondent à plusieurs objectifs :

Python est un langage déjà utilisé dans plusieurs **cours** de la licence. Il permet d'approfondir des notions vues en algorithmique et en structures de données tout en restant accessible pour le groupe.

Python, comme langage unique pour le projet, permet une **architecture légère**, diminuant la complexité globale du projet tout en respectant les exigences du cours.

Le but est aussi d'éviter de **potentielles erreurs de configuration** le jour de l'épreuve. Facile et rapide à exécuter, le projet pourra être lancé sur **Kubuntu** sans nécessiter de configuration lourde (par exemple base de **données**, authentification, dépendances complexes).

Le projet ne nécessite ni gestion de plusieurs utilisateurs ni persistance complexe des données ; un **stockage local** est donc suffisant pour répondre aux objectifs fonctionnels (type fichiers JSON).

Cette architecture **pourrait** permettre d'enrichir le projet ultérieurement (amélioration de l'interface, nouvelles fonctionnalités) sans remettre en cause la base existante.

5.1 Architecture cible

Frontend (Python) :

- écrans de gestion (CRUD) des tâches;
- visualisation du planning (liste + Gantt simplifié);
- validation côté client (champs requis, durées positives, dépendances cohérentes).

Backend (Python) :

- API REST (JSON) pour les tâches, dépendances et calcul de planning;
- logique métier (construction du graphe, détection de cycles, tri topologique, calcul des dates);
- gestion des erreurs (messages explicites pour le frontend).

Stockage local (JSON) :

- fichier JSON unique contenant tâches et dépendances;
- chargement en mémoire au démarrage;
- sauvegarde après chaque modification;
- solution robuste pour la démo et sans dépendance externe.

5.2 Modélisation des données (format JSON)

Les données sont stockées dans un fichier tasks.json.

Structure proposée :

```
{  
  "tasks": [  
    {  
      "id": 1,  
      "name": "T1",  
      "duration": 3,  
      "priority": 1,  
      "depends_on": []  
    },  
  ],  
}
```



```

{
  "id": 2,
  "name": "T2",
  "duration": 2,
  "priority": 2,
  "depends_on": [1]
}
]
}

```

Remarque

Les champs start et end ne sont pas stockés mais calculés dynamiquement par le backend lors de l'appel à /plan.

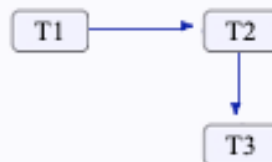
5.3 Algorithmes et règles de calcul

Méthode 1 — Construction du graphe

Idée : représenter les tâches comme un graphe orienté (DAG attendu).

— Sommets = tâches.

— Arêtes : depends_on → task.

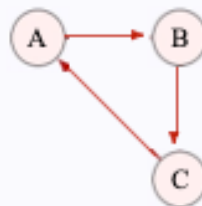


Méthode 2 — Détection de cycles

But : refuser tout cycle (sinon pas d'ordonnancement possible).

— Option A : DFS avec marquage (blanc/gris/noir).

— Option B : Kahn (si des nœuds restent, il y a cycle).



Erreur attendue : « Cycle détecté : A → B → C → A ».

Méthode 3 — Tri topologique (algorithme de Kahn)

Principe : répéter : prendre un nœud sans prédécesseur, l'ajouter à l'ordre, puis retirer ses arêtes.

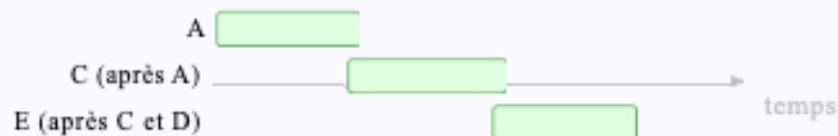
Un ordre valide est par exemple : A, B, C, D, E . Table de progression :

Étape	Nœuds sans prédécesseur	Sortie (ordre)
0 1	$\{A, B\}$	
2 3	choisir A	$\emptyset$
4 5	$\{B, C\}$	$[A]$
6 7	choisir B	$[A, B]$
	$\{C, D\}$	$[A, B]$
	choisir C puis D	$[A, B, C, D]$
	$\{E\}$	$[A, B, C, D]$
	choisir E	$[A, B, C, D, E]$

Méthode 4 — Calcul des dates (planning le plus tôt)

Règle : pour chaque tâche t ,

$$\text{début}(t) = \max_{p \in \text{pré}(t)} \text{fin}(p), \quad \text{fin}(t) = \text{début}(t) + \text{durée}(t).$$



Note : les tâches prêtes peuvent être ordonnées par priorité puis par identifiant pour un résultat déterministe.

5.4 API (Flask léger)

- GET /tasks : retourne le JSON courant
- POST /tasks : ajoute une tâche et sauvegarde le fichier
- PUT /tasks/<id> : modifie une tâche
- DELETE /tasks/<id> : supprime une tâche
- POST /plan : calcule ordre et dates, renvoie erreur si cycle

6. Fonctionnement général

1 - Saisie de données par l'utilisateur

L'utilisateur crée des tâches (voir 5.1 Modélisation de données) via l'interface.

2 - Stockage des données

Les données saisies par l'utilisateur sont stockées dans des fichiers JSON.

3 - Traitement des données

Le back-end construit le graphe à partir des données, puis vérifie l'absence de cycles avant de calculer l'ordre d'exécution et générer le planning. (voir 5.2

Algorithmes utilisés)

4 - Affichage du résultat

Le front-end affiche la liste ordonnée des tâches, accompagnée d'un mini diagramme de Gantt simplifié et potentiellement une synthèse (voir 3.3 Fonctionnalités optionnelles).

7. Répartition du travail

- **Backend (API + algorithmes)** : Paul, Rémy
- **Frontend (UI + intégration API)** : Ayoub, Mathilde
- **Qualité (tests, CI, revue)** : responsabilité partagée (revues croisées systématiques)
- **Documentation (rapport)** : responsabilité partagée (un chapitre par binôme *À voir avec M.Kislin*)

8. Tests et validation

8.1 Stratégie de tests

Les tests servent à vérifier le bon fonctionnement du programme **conformément aux spécifications** définies dans ce cahier des charges.

Ils porteront notamment sur :

- le bon fonctionnement des algorithmes de planification
- le respect des dépendances entre les tâches
- la détection d'erreurs
- la cohérence du planning généré

Des **tests unitaires simples** pourront être mis en place afin de valider certaines fonctions clés (ordre d'exécution des tâches, gestion des dépendances).

8.2 Validation du projet

La validation du projet reposera sur :

- l'exécution complète du programme dans un environnement Linux (**Kubuntu**)
- la capacité à lancer le projet simplement, sans configuration complexe
- la démonstration des fonctionnalités lors de la soutenance
- la vérification du code par l'enseignant

Une attention particulière sera portée à la **stabilité du programme**, afin d'assurer son bon fonctionnement lors de la soutenance finale.

9. Livrables attendus

Les livrables du projet comprennent :

1. Code source complet

- Backend : implémentation des algorithmes de planification, gestion des dépendances et calcul des dates.
- Frontend : interface utilisateur simple et intuitive, affichage du planning sous forme de liste et de mini diagramme de Gantt.
- Documentation technique : commentaires dans le code, instructions pour l'installation et l'exécution du projet.

2. Fichiers JSON de données et fichiers de test

- Fichiers JSON contenant des exemples de tâches avec dépendances et priorités.
- Schéma et données de test permettant de simuler différents scénarios de planification.

3. Rapport écrit

- Description détaillée des choix techniques effectués.
- Présentation des algorithmes utilisés (détection de cycles, tri topologique, calcul des dates).
- Difficultés rencontrées et solutions mises en place.
- Analyse de la structure du projet et de son architecture globale.

4. Démonstration fonctionnelle

- Exécution complète du programme dans un environnement Linux (Kubuntu).
- Présentation des fonctionnalités principales lors de la soutenance.